SIMATS ENGINEERING
ASSESSMENT TOOL 1

COURSE CODE: CSA14 **COURSE NAME: Compiler Design**

COURSE OUTCOME COVERED

CO2: Construct top-down and bottom up parsers using CFG for parsing conflicts and error recovery for efficient syntax tree generation. *(BL3)*

Assessment Tool Weightage: 40%

Duration :60 Mins
Off Class
QUESTIONS: 5 **Total Marks:50**

Annexure A

DESIGN TASK

Code of Conduct:

I Karanya S (192421309) *(Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.*
I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

Karanya S

SIMATS
ENGINEERING

SIMATS
Saveetha Institute of Medical And Technical Sciences
(Declared as Deemed to be University under Section 3 of UGC Act 1956)

Name: Karanya S

Course Code and Course Name: CSA1403

COMPILER DESIGN

Register No: 192421309

Date: 06/08/2026

Table of Contents

1. Introduction

A compiler translates a program written in a high-level source language into an equivalent program in a target language, typically machine code. The front end of a compiler consists of two principal phases: lexical analysis and syntax analysis. The lexical analyzer (scanner) scans the source text and groups characters into meaningful sequences called lexemes, producing a stream of tokens. The syntax analyzer (parser) then checks whether the sequence of tokens conforms to the grammatical rules of the language, as specified by a context-free grammar, and reports the input as syntactically Accepted or Rejected.

This laboratory record documents the design and implementation of five parsers, each built using the classical Lex/Yacc toolchain. Every experiment follows a common methodology: a context-free grammar is first specified for the construct under study; a lexical analyzer is written in Lex to tokenize the input; a parser is written in Yacc using the corresponding grammar productions; and the combined tool is compiled and executed to validate sample inputs, displaying Accepted for syntactically valid input and Rejected, together with a diagnostic message, for invalid input.

The five constructs studied are: (1) a C-style return statement, (2) an SQL INSERT statement, (3) a simple JSON object, (4) a single-dimensional array declaration, and (5) a two-dimensional array declaration. Together these exercises illustrate how Yacc handles keyword-based statement forms, comma-separated value/argument lists, nested bracket structures, and declaration syntax involving type specifiers and array dimensions.

2. Software and Hardware Requirements

The following software environment was used to design, compile and test all five parsers described in this record.

Software	**Purpose**
Windows 10 / 11	Host operating system
Windows Subsystem for Linux (WSL)	Provides a Linux environment on Windows
Ubuntu 24.04 LTS (WSL distribution)	Linux operating system used to run Flex and Bison
Visual Studio Code (VS Code)	Source code editor
WSL Extension (VS Code)	Connects VS Code to the Ubuntu environment
GNU Flex (Lex)	Lexical analyzer generator
GNU Bison (Yacc)	Parser generator
GCC (GNU Compiler Collection)	Compiles the C code generated by Flex and Bison
Terminal (Ubuntu / VS Code terminal)	Used to execute the compilation and run commands

Hardware requirement: any personal computer capable of running Windows 10/11 with WSL2 enabled (minimum 4 GB RAM, 64-bit processor with virtualization support) is sufficient, since all five experiments are lightweight command-line programs.

3. Overview of Lex and Yacc

Lex (implemented here using the GNU Flex clone) reads a specification file containing regular-expression patterns paired with C action code, and generates a C source file, lex.yy.c, containing a function yylex() that returns one token at a time to the caller. Yacc (implemented using GNU Bison) reads a specification file containing context-free grammar productions annotated with semantic actions, and generates a C source file, typically y.tab.c, containing a function yyparse() that repeatedly calls yylex() to obtain tokens and applies a bottom-up LALR(1) shift-reduce parsing algorithm to determine whether the token stream can be derived from the start symbol of the grammar.

In every experiment below, the Lex specification is compiled first so that the token definitions in the generated y.tab.h header (produced by Bison with the -d option) are available to the scanner. The general build sequence used throughout this record is: bison -d parser.y, flex lexer.l, and gcc lex.yy.c y.tab.c -o parser -lfl, followed by execution of the resulting parser executable with the given sample input.

Experiment 1: Recognition of Return Statements

1. Aim

To design and implement a lexical analyzer and parser, using Lex and Yacc, that recognizes and validates a C-style return statement of the form return E ; , where E may be an identifier, a numeric literal, or an additive expression combining them, and reports Accepted or Rejected.

2. Grammar

S → return E ;
E → id
E → num
(extension for arithmetic expression, required by the sample input) E → E + E

3. Algorithm / Procedure

1. Define a keyword token RETURN for "return", together with tokens ID for identifiers, NUMBER for numeric literals, PLUS for the '+' operator, and SEMICOLON for ';' in the Lex specification. Place the RETURN keyword rule before the general identifier rule so that the reserved word is not misclassified as an identifier.

2. Define the grammar productions for S and E in the Yacc specification. The base grammar allows E to reduce directly to id or num; a left-recursive alternative E → E PLUS E is added so that additive expressions such as a+10, as required by the given sample input, can be parsed.

3. Attach a semantic action to the S production that prints "Accepted" once the full pattern return E ; has been successfully reduced.

4. Attach a yyerror() routine that prints "Rejected" with a diagnostic message whenever the parser encounters a token sequence with no valid shift or reduce action, such as a return statement with no expression or a missing semicolon.

5. Compile the Lex and Yacc files together and test with the sample input return a+10;

4. Lexical Analyzer (Lex Specification)

```
%{
#include "parser.tab.h"
#include <stdio.h>
%}
%%
"return"                    { return RETURN; }
[A-Za-z_][A-Za-z0-9_]*      { return ID; }
[0-9]+                      { return NUMBER; }
"+"                         { return PLUS; }
";"                         { return SEMICOLON; }
[ \t\n]+                    ;
.                           { return yytext[0]; }
%%
int yywrap()
```

```
{
    return 1;
}
```

5. Parser (Yacc Specification)

```
%{
#include<stdio.h>
#include<stdlib.h>
int yylex();
void yyerror(const char *s);
%}
%token RETURN ID NUMBER PLUS SEMICOLON
%left PLUS
%%
S :
    RETURN E SEMICOLON
    {
        printf("\nAccepted\n");
    }
    ;
E :
    E PLUS E
    | ID
    | NUMBER
    ;
%%
void yyerror(const char *s)
{
    printf("\nRejected\n");
}
int main()
{
    printf("Enter Return Statement:\n");
    yyparse();
    return 0;
}
```

6. Compilation and Execution Steps

```
$ bison -d return_parser.y
$ flex return_lexer.l
$ gcc lex.yy.c y.tab.c -o return_parser -lfl
$ ./return_parser
Enter return statement:
return a+10;
```

7. Sample Input and Output

Input: return a+10;

Tokenized: RETURN ID('a') '+' NUMBER(10) ';'

Derivation: S -> return E ; -> return E+E ; -> return id+num ;

Output: Accepted : valid return statement

For an erroneous input such as return ; (a return statement with no expression), the parser invokes yyerror() and prints: Rejected : syntax error - syntax error

8. Result

The parser correctly identifies well-formed return statements consisting of an identifier, a numeric literal, or an additive combination of the two, terminated with a semicolon, and rejects malformed statements, thereby verifying the given grammar.

Experiment 2: Recognition of SQL INSERT Statements

1. Aim

To design and implement a lexical analyzer and parser, using Lex and Yacc, that recognizes and validates an SQL INSERT statement of the form INSERT INTO id VALUES (ValueList) ; and reports Accepted or Rejected.

2. Grammar

S → INSERT INTO id VALUES (ValueList) ;
ValueList → value
ValueList → value , ValueList
value → NUM | STRING

3. Algorithm / Procedure

1. Define keyword tokens INSERT, INTO and VALUES, a token ID for the table name, and tokens NUMBER and STRING for numeric and single-quoted string values, together with the symbols (,), comma and semicolon, in the Lex specification. Place the keyword rules before the general identifier rule.

2. Define the grammar productions for S and ValueList in the Yacc specification, with value reducing to either a number or a string token.

3. Attach a semantic action to the S production that prints "Accepted" once the full pattern INSERT INTO id VALUES (ValueList) ; has been successfully reduced.

4. Attach a yyerror() routine that prints "Rejected" with a diagnostic message whenever the parser encounters a malformed statement, such as a missing keyword, missing parenthesis, or missing semicolon.

5. Compile the Lex and Yacc files together and test with the sample input INSERT INTO Student VALUES(101,'John');

4. Lexical Analyzer (Lex Specification)

```
%{
#include "parser.tab.h"
#include <stdio.h>
%}
%%
"INSERT"                    { return INSERT; }
"INTO"                      { return INTO; }
"VALUES"                    { return VALUES; }
[A-Za-z_][A-Za-z0-9_]*      { return ID; }
[0-9]+                      { return NUMBER; }
\'[^\']*\'                   { return STRING; }
"("                         { return LPAREN; }
")"                         { return RPAREN; }
","                         { return COMMA; }
";"                         { return SEMICOLON; }
[ \t\n]+                    ;
```

```
.                           { return yytext[0]; }
%%
int yywrap()
{
    return 1;
}
```

5. Parser (Yacc Specification)

```
%{
#include<stdio.h>
#include<stdlib.h>
int yylex();
void yyerror(const char *s);
%}
%token INSERT INTO VALUES ID NUMBER STRING
%token LPAREN RPAREN COMMA SEMICOLON
%%
S :
    INSERT INTO ID VALUES LPAREN ValueList RPAREN SEMICOLON
    {
        printf("\nAccepted\n");
    }
    ;
ValueList :
    Value
    | Value COMMA ValueList
    ;
Value :
    NUMBER
    | STRING
    ;
%%
void yyerror(const char *s)
{
    printf("\nRejected\n");
}
int main()
{
    printf("Enter SQL INSERT Statement:\n");
    yyparse();
    return 0;
}
```

6. Compilation and Execution Steps

```
$ bison -d sql_parser.y
$ flex sql_lexer.l
$ gcc lex.yy.c y.tab.c -o sql_parser -lfl
$ ./sql_parser
Enter SQL INSERT statement:
INSERT INTO Student VALUES(101,'John');
```

7. Sample Input and Output

Input: INSERT INTO Student VALUES(101,'John');

Tokenized: INSERT INTO ID('Student') VALUES '(' NUMBER(101) ',' STRING('John') ')' ';'

Output: Accepted : valid SQL INSERT statement

For an erroneous input such as INSERT INTO Student VALUES(101,'John') (missing terminating semicolon), the parser invokes yyerror() and prints: Rejected : syntax error - syntax error

8. Result

The parser correctly identifies well-formed INSERT statements consisting of a table identifier followed by a parenthesised, comma-separated list of numeric and string values terminated with a semicolon, and rejects malformed statements, thereby verifying the given grammar.

Experiment 3: Recognition of Simple JSON Objects

1. Aim

To design and implement a lexical analyzer and parser, using Lex and Yacc, that recognizes and validates a simple JSON object of the form { Members }, where each member is a string key mapped to a string or numeric value, and reports Accepted or Rejected.

2. Grammar

Object → { Members }
Members → Pair
Members → Pair , Members
Pair → string : value
(extension for value type, required by the sample input) value → STRING | NUMBER

3. Algorithm / Procedure

1. Define a token STRING for double-quoted key/value text and a token NUMBER for numeric literals, together with the symbols {, }, colon and comma, in the Lex specification.

2. Define the grammar productions for Object, Members, Pair and Value in the Yacc specification, with value reducing to either a string or a number so that both the "John" and 20 values in the sample input can be parsed.

3. Attach a semantic action to the start production that prints "Accepted" once the complete Object has been successfully reduced.

4. Attach a yyerror() routine that prints "Rejected" with a diagnostic message whenever the parser encounters a malformed object, such as a missing colon, a trailing comma, or an unbalanced brace.

5. Compile the Lex and Yacc files together and test with the sample input {"name":"John","age":20}

4. Lexical Analyzer (Lex Specification)

```
%{
#include "parser.tab.h"
#include <stdio.h>
%}
%%
\"[^\"]*\"                  { return STRING; }
[0-9]+                      { return NUMBER; }
"{"                         { return LBRACE; }
"}"                         { return RBRACE; }
":"                         { return COLON; }
","                         { return COMMA; }
[ \t\n]+                    ;
.                           { return yytext[0]; }
%%
int yywrap()
```

```
{
    return 1;
}
```

5. Parser (Yacc Specification)

```
%{
#include<stdio.h>
#include<stdlib.h>
int yylex();
void yyerror(const char *s);
%}
%token STRING NUMBER LBRACE RBRACE COLON COMMA
%%
Start :
    Object
    {
        printf("\nAccepted\n");
    }
    ;
Object :
    LBRACE Members RBRACE
    ;
Members :
    Pair
    | Pair COMMA Members
    ;
Pair :
    STRING COLON Value
    ;
Value :
    STRING
    | NUMBER
    ;
%%
void yyerror(const char *s)
{
    printf("\nRejected\n");
}
int main()
{
    printf("Enter JSON Object:\n");
    yyparse();
    return 0;
}
```

6. Compilation and Execution Steps

```
$ bison -d json_parser.y
$ flex json_lexer.l
$ gcc lex.yy.c y.tab.c -o json_parser -lfl
$ ./json_parser
```

```
Enter JSON object:
{"name":"John","age":20}
```

7. Sample Input and Output

Input: {"name":"John","age":20}

Tokenized: '{' STRING("name") ':' STRING("John") ',' STRING("age") ':' NUMBER(20) '}'

Derivation: Object -> {Members} -> {Pair,Members} -> {Pair,Pair}

Output: Accepted : valid JSON object

For an erroneous input such as {"name":"John",} (a trailing comma with no following pair), the parser invokes yyerror() and prints: Rejected : syntax error - syntax error

8. Result

The parser correctly identifies well-formed JSON objects consisting of a comma-separated list of string-keyed pairs enclosed in braces, with values of either string or numeric type, and rejects malformed objects, thereby verifying the given grammar.

Experiment 4: Recognition of Array Declarations

1. Aim

To design and implement a lexical analyzer and parser, using Lex and Yacc, that recognizes and validates a single-dimensional array declaration of the form Type id [num] ; and reports Accepted or Rejected.

2. Grammar

D → Type id [num] ;
Type → int | float | char

3. Algorithm / Procedure

1. Define keyword tokens INT, FLOAT and CHAR_TYPE, together with tokens for identifiers (ID), numeric literals (NUMBER), and the symbols [,] and semicolon, in the Lex specification. Ensure the Lex rule for keywords is placed before the general identifier rule so that reserved words are not misclassified as identifiers.

2. Define the grammar productions for D and Type in the Yacc specification.

3. Attach a semantic action to the D production that prints "Accepted" once the pattern Type id [num] ; is fully reduced.

4. Attach a yyerror() routine that prints "Rejected" with a diagnostic message whenever the parser encounters a malformed declaration, such as a missing array size or a missing semicolon.

5. Compile the Lex and Yacc files together and test with the sample input float marks[100];

4. Lexical Analyzer (Lex Specification)

```
%{
#include "parser.tab.h"
#include <stdio.h>
%}
%%
"int"                       { return INT; }
"float"                     { return FLOAT; }
"char"                      { return CHAR_TYPE; }
[A-Za-z_][A-Za-z0-9_]*      { return ID; }
[0-9]+                      { return NUMBER; }
"["                         { return LBRACKET; }
"]"                         { return RBRACKET; }
";"                         { return SEMICOLON; }
[ \t\n]+                    ;
.                           { return yytext[0]; }
%%
int yywrap()
{
    return 1;
}
```

5. Parser (Yacc Specification)

```
%{
#include<stdio.h>
#include<stdlib.h>
int yylex();
void yyerror(const char *s);
%}
%token INT FLOAT CHAR_TYPE ID NUMBER
%token LBRACKET RBRACKET SEMICOLON
%%
D :
    Type ID LBRACKET NUMBER RBRACKET SEMICOLON
    {
        printf("\nAccepted\n");
    }
    ;
Type :
    INT
    | FLOAT
    | CHAR_TYPE
    ;
%%
void yyerror(const char *s)
{
    printf("\nRejected\n");
}
int main()
{
    printf("Enter Array Declaration:\n");
    yyparse();
    return 0;
}
```

6. Compilation and Execution Steps

```
$ bison -d array_parser.y
$ flex array_lexer.l
$ gcc lex.yy.c y.tab.c -o array_parser -lfl
$ ./array_parser
Enter array declaration:
float marks[100];
```

7. Sample Input and Output

Input: float marks[100];

Tokenized: FLOAT ID('marks') '[' NUMBER(100) ']' ';'

Output: Accepted : valid array declaration

For an erroneous input such as float marks[]; (a missing array size), the parser invokes yyerror() and prints: Rejected : syntax error - syntax error

8. Result

The parser correctly identifies well-formed single-dimensional array declarations for the int, float and char types, consisting of a type specifier, an identifier, a bracketed numeric size, and a terminating semicolon, and rejects malformed declarations, thereby verifying the given grammar.

Experiment 5: Recognition of Multidimensional Array Declarations

1. Aim

To design and implement a lexical analyzer and parser, using Lex and Yacc, that recognizes and validates a two-dimensional array declaration of the form Type id [num] [num] ; and reports Accepted or Rejected.

2. Grammar

D → Type id [num] [num] ;
Type → int | double

3. Algorithm / Procedure

1. Define keyword tokens INT and DOUBLE, together with tokens for identifiers (ID), numeric literals (NUMBER), and the symbols [,] and semicolon, in the Lex specification. Ensure the Lex rule for keywords is placed before the general identifier rule.

2. Define the grammar productions for D and Type in the Yacc specification, repeating the bracketed-dimension pattern twice to capture the row and column sizes.

3. Attach a semantic action to the D production that prints "Accepted" once the pattern Type id [num] [num] ; is fully reduced.

4. Attach a yyerror() routine that prints "Rejected" with a diagnostic message whenever the parser encounters a malformed declaration, such as a missing second dimension or a missing semicolon.

5. Compile the Lex and Yacc files together and test with the sample input double matrix[5][10];

4. Lexical Analyzer (Lex Specification)

```
%{
#include "parser.tab.h"
#include <stdio.h>
%}
%%
"int"                       { return INT; }
"double"                    { return DOUBLE; }
[A-Za-z_][A-Za-z0-9_]*      { return ID; }
[0-9]+                      { return NUMBER; }
"["                         { return LBRACKET; }
"]"                         { return RBRACKET; }
";"                         { return SEMICOLON; }
[ \t\n]+                    ;
.                           { return yytext[0]; }
%%
int yywrap()
{
    return 1;
}
```

5. Parser (Yacc Specification)

```
%{
#include<stdio.h>
#include<stdlib.h>
int yylex();
void yyerror(const char *s);
%}
%token INT DOUBLE ID NUMBER
%token LBRACKET RBRACKET SEMICOLON
%%
D :
    Type ID LBRACKET NUMBER RBRACKET LBRACKET NUMBER RBRACKET SEMICOLON
    {
        printf("\nAccepted\n");
    }
    ;
Type :
    INT
    | DOUBLE
    ;
%%
void yyerror(const char *s)
{
    printf("\nRejected\n");
}
int main()
{
    printf("Enter Multidimensional Array Declaration:\n");
    yyparse();
    return 0;
}
```

6. Compilation and Execution Steps

```
$ bison -d matrix_parser.y
$ flex matrix_lexer.l
$ gcc lex.yy.c y.tab.c -o matrix_parser -lfl
$ ./matrix_parser
Enter multidimensional array declaration:
double matrix[5][10];
```

7. Sample Input and Output

Input: double matrix[5][10];

Tokenized: DOUBLE ID('matrix') '[' NUMBER(5) ']' '[' NUMBER(10) ']' ';'

Output: Accepted : valid multidimensional array declaration

For an erroneous input such as double matrix[5]; (a missing second dimension), the parser invokes yyerror() and prints: Rejected : syntax error - syntax error

8. Result

The parser correctly identifies well-formed two-dimensional array declarations for the int and double types, consisting of a type specifier, an identifier, and two bracketed numeric dimensions terminated with a semicolon, and rejects declarations that omit either dimension, thereby verifying the given grammar.

9. Conclusion

This laboratory exercise demonstrated the complete workflow of building small, special-purpose language recognizers using the Lex and Yacc (Flex/Bison) toolchain on a WSL-based Ubuntu 24.04 environment. Five distinct grammatical constructs commonly found in programming and data-interchange languages — return statements, SQL INSERT statements, JSON objects, single-dimensional array declarations, and two-dimensional array declarations — were each specified as a context-free grammar, tokenized by a hand-written Lex scanner, and parsed by a corresponding Yacc bottom-up parser. In every case, the parser was able to correctly display Accepted for syntactically valid sample input and Rejected, with an appropriate diagnostic, for erroneous input, thereby verifying that the lexical analyzer and parser together implement the specified grammar faithfully.

The exercises also illustrate recurring practical concerns in parser construction: extending a minimal grammar with small, conservative additions where a sample input requires them (as with the additive return-expression and the typed JSON value), ensuring keyword tokens are matched ahead of the general identifier rule, and correctly handling nested or repeated bracketed and parenthesised structures such as SQL argument lists, JSON member lists, and multidimensional array dimensions.